

Interfaccia HTML

Introduzione ai tag HTML e loro utilizzo.

```
/**  
* @author Rocco Mazzeo  
* @email rocco.mazzeo@gmail.com  
* @linkedin https://www.linkedin.com/in/roccomazzeo  
*/
```

Cos'è HTML?

HTML (HyperText Markup Language) è il linguaggio standard per creare pagine web.

Storia di HTML

HTML nasce nel 1991 grazie a Tim Berners-Lee, che lo sviluppa per condividere documenti scientifici tramite il World Wide Web. La prima versione era molto semplice e includeva pochi tag. Nel tempo, HTML si è evoluto attraverso diverse versioni (HTML 2.0, 3.2, 4.01) fino ad arrivare a HTML5, introdotto nel 2014, che ha aggiunto nuove funzionalità per multimedia, grafica e interattività. Oggi HTML è uno standard fondamentale per la creazione di siti web moderni.

Struttura di base

```
<!DOCTYPE html>
<html>
  <head>
    <title>Titolo Pagina</title>
  </head>
  <body>
    Contenuto qui
  </body>
</html>
```

Editor HTML online per test

Puoi provare e testare il codice HTML direttamente online usando editor interattivi. Ecco alcuni strumenti utili:

- [JSFiddle](#)

Editor online per testare codice web e condividerlo facilmente.

- [PlayCode](#)

Ambiente semplice per esperimenti veloci con HTML, CSS e JS.

Consiglio: Usa questi editor per esercitarti, vedere subito il risultato e condividere il codice con altri.

Come sono fatti i tag HTML

Un tag HTML è racchiuso tra parentesi angolari `< >`. La maggior parte dei tag ha una versione di apertura `<tag>` e una di chiusura `</tag>`. Alcuni tag, come `<img>` o `<br>`, sono auto-chiudenti.

Attributi HTML

I tag possono avere attributi, che aggiungono informazioni o modificano il comportamento del tag. Gli attributi sono scritti all'interno del tag di apertura, nella forma `nome="valore"`.

```
<tag-nome attributo1="valore1" attributo2="valore2">Contenuto</tag-nome>
```

- `<tag-nome>` : nome del tag HTML.
- `attributo="valore"` : attributi opzionali che modificano il comportamento del tag.
- `Contenuto` : ciò che viene visualizzato tra il tag di apertura e quello di chiusura (se previsto).

Esempio:

```
<a href="https://www.example.com" target="_blank">Visita Example</a>  

```

- `href` e `target` sono attributi del tag `<a>` .
- `src` e `alt` sono attributi del tag `<img>` .

Attributi principali dei tag HTML

Ecco una panoramica degli attributi più importanti per i principali tag HTML e il loro significato:

Tag	Attributi principali	Significato
<code><html></code>	<code>lang</code> , <code>dir</code>	Lingua del documento, direzione del testo
<code><head></code>	Nessuno specifico	Contenitore di metadati
<code><title></code>	Nessuno	Titolo della pagina
<code><body></code>	Nessuno	Contenuto visibile
<code><a></code>	<code>href</code> , <code>target</code> , <code>title</code>	URL destinazione, dove aprire il link, descrizione tooltip
<code></code>	<code>src</code> , <code>alt</code> , <code>width</code> , <code>height</code>	Percorso immagine, testo alternativo, dimensioni

Tag	Attributi principali	Significato
<code><form></code>	<code>action</code> , <code>method</code> , <code>enctype</code>	URL invio dati, metodo invio, tipo codifica dati
<code><input></code>	<code>type</code> , <code>name</code> , <code>id</code> , <code>value</code> , <code>placeholder</code> , <code>required</code> , <code>readonly</code> , <code>disabled</code>	Tipo campo, nome, identificatore, valore, suggerimento, obbligatorietà, sola lettura, disabilitato
<code><label></code>	<code>for</code>	Associa etichetta a campo tramite id
<code><select></code>	<code>name</code> , <code>id</code> , <code>multiple</code>	Nome, identificatore, selezione multipla
<code><option></code>	<code>value</code> , <code>selected</code>	Valore inviato, selezionato di default
<code><textarea></code>	<code>name</code> , <code>id</code> , <code>rows</code> , <code>cols</code> , <code>placeholder</code>	Nome, identificatore, dimensioni, suggerimento

Tag	Attributi principali	Significato
<code><button></code>	<code>type</code> , <code>name</code> , <code>value</code> , <code>disabled</code>	Tipo pulsante, nome, valore, disabilitato
<code><table></code>	<code>border</code> , <code>cellpadding</code> , <code>cellspacing</code>	Bordo, spaziatura celle
<code><tr></code> , <code><td></code> , <code><th></code>	<code>colspan</code> , <code>rowspan</code>	Celle unite orizzontalmente/verticalmente
<code><link></code>	<code>rel</code> , <code>href</code> , <code>type</code>	Relazione, percorso risorsa, tipo file
<code><script></code>	<code>src</code> , <code>type</code> , <code>async</code> , <code>defer</code>	Percorso script, tipo, caricamento asincrono/differito
<code><meta></code>	<code>charset</code> , <code>name</code> , <code>content</code>	Codifica caratteri, nome metadato, valore

Tag	Attributi principali	Significato
<code><div></code> , <code></code>	<code>id</code> , <code>class</code> , <code>style</code>	Identificatore, classi CSS, stili inline
<code><audio></code> , <code><video></code>	<code>src</code> , <code>controls</code> , <code>autoplay</code> , <code>loop</code>	Percorso file, controlli, riproduzione automatica, ripeti
<code><iframe></code>	<code>src</code> , <code>width</code> , <code>height</code> , <code>title</code>	Percorso pagina, dimensioni, descrizione

Nota: Quasi tutti i tag accettano attributi globali come `id`, `class`, `style`, `title`, `tabindex`, `hidden`.

Esempio di utilizzo attributi

```
<a href="https://www.example.com" target="_blank" title="Vai a Example"
  >Visita Example</a
>

<input
  type="email"
  name="user_email"
  placeholder="Inserisci la tua email"
  required
/>
```

Gli attributi più comuni sono: `id`, `class`, `style`, `src`, `href`, `alt`, `title`, `value`, `type`, ecc.

Come validare i file HTML

Online

Puoi validare i tuoi file HTML tramite il [W3C Markup Validation Service](#). Basta caricare il file o inserire l'URL della pagina per ricevere un report sugli errori e sulle best practice.

In locale con VS Code

1. Installa l'estensione "**HTMLHint**" da Visual Studio Code Marketplace.
2. Dopo l'installazione, HTMLHint analizzerà automaticamente i tuoi file HTML e segnalerà eventuali errori o avvisi direttamente nell'editor.
3. Puoi personalizzare le regole di validazione creando un file `.htmlhintrc` nella cartella del progetto.

Consiglio: Validare regolarmente il codice aiuta a mantenere la compatibilità e la qualità delle pagine web.

Compatibilità dei tag HTML

Non tutti i tag e attributi HTML sono supportati allo stesso modo da tutti i browser. Prima di utilizzare funzionalità avanzate, è importante verificare la compatibilità per evitare problemi di visualizzazione.

Come verificare la compatibilità

Puoi usare siti come [Can I use](#) per controllare quali browser supportano un determinato tag, attributo o tecnologia web.

Consiglio: Consulta sempre la compatibilità prima di usare nuove funzionalità, soprattutto se il sito deve funzionare su dispositivi e browser diversi.

Documentazione online

[w3schools](#) <- Ottima risorsa per imparare i tag HTML con esempi pratici.>

[spec.whatwg.org](#) <- Specifica ufficiale HTML5.>

Tag `<html>`

Definisce l'inizio e la fine di un documento HTML.

Esempio e dettagli aggiuntivi

```
<html lang="it" dir="ltr">
  <!-- Attributi: lang specifica la lingua, dir la direzione del testo -->
  <head>
    <meta charset="UTF-8" />
    <title>Esempio HTML</title>
  </head>
  <body>
    <h1>Ciao!</h1>
  </body>
</html>
```

Note sugli attributi del tag `<html>`

- Il tag `<html>` può avere **più attributi** (cardinalità: zero o più), come `lang` (lingua del documento) e `dir` (direzione del testo).
- Alcuni attributi sono **obbligatori** in certi contesti (es. `alt` per `<img>`), altri sono **opzionali**.
- La cardinalità degli attributi dipende dal tag: alcuni accettano solo attributi specifici, altri permettono attributi globali (`id`, `class`, `style`, ecc.).
- I valori degli attributi devono essere racchiusi tra virgolette.

Tag `<head>`

Contiene metadati, titoli, link a CSS e script.

Esempi di contenuto nel `<head>`

```
<head>
  <meta charset="UTF-8" />
  <title>Il mio sito</title>
  <link rel="stylesheet" href="stile.css" />
  <script src="script.js"></script>
</head>
```

- `<meta>` definisce i metadati (es. charset).
- `<title>` imposta il titolo della pagina.
- `<link>` collega un foglio di stile CSS.
- `<script>` collega o inserisce codice JavaScript.

Tag `<title>`

Imposta il titolo della pagina nel browser.

Esempi di utilizzo

```
<title>Corso di Programmazione Web</title>
```

- Il testo tra i tag `<title>` appare nella scheda del browser.
- È importante per l'accessibilità e il posizionamento nei motori di ricerca (SEO).
- Ogni pagina dovrebbe avere un titolo unico e descrittivo.

Cos'è la SEO?

La SEO (Search Engine Optimization) è l'insieme di tecniche che aiutano le pagine web a essere trovate più facilmente dai motori di ricerca come Google.

Perché è importante?

Se una pagina è ottimizzata per la SEO, sarà più visibile nei risultati di ricerca e avrà più possibilità di essere visitata dagli utenti.

Cosa c'entra l'HTML?

Scrivere il codice HTML in modo corretto e ordinato aiuta i motori di ricerca a capire meglio il contenuto della pagina. Ad esempio, usare titoli chiari, descrizioni e testi alternativi per le immagini sono semplici accorgimenti che migliorano la SEO.

Tag `<body>`

Contiene tutto il contenuto visibile della pagina.

Il tag `<body>` è la sezione principale del documento HTML dove vengono inseriti tutti gli elementi visibili all'utente: testi, immagini, link, moduli, tabelle, ecc. Tutto ciò che appare nella pagina web viene scritto all'interno del `<body>`.

Esempio

```
<body>  
  <h1>Benvenuto!</h1>  
  <p>Questa è la parte visibile della pagina.</p>  
    
</body>
```

- Il tag `<body>` può contenere qualsiasi altro tag HTML che rappresenta contenuto o struttura della pagina.

Tag `<body>` - Dettagli aggiuntivi

- In ogni documento HTML deve esserci **un solo** `<body>`, non sono ammessi più tag `<body>`.
- Gli stili CSS e gli script JavaScript possono agire sugli elementi all'interno del `<body>`.
- Il `<body>` viene visualizzato dopo che il browser ha caricato i metadati definiti nel `<head>`.

Tag `<h1>` - `<h6>`

Intestazioni di vari livelli.

```
<h1>Titolo principale</h1>  
<h2>Sottotitolo</h2>  
<h3>Sottotitolo di terzo livello</h3>  
<h4>Sottotitolo di quarto livello</h4>  
<h5>Sottotitolo di quinto livello</h5>  
<h6>Sottotitolo di sesto livello</h6>
```


Tag `<p>`

Definisce un paragrafo.

```
<p>Questo è un paragrafo.</p>
```

Tag `<br>`

Va a capo (line break).

```
Testo<br />Nuova linea
```

Tag `<hr>`

Linea orizzontale.

```
<hr />
```

Tag `<a>`

Collegamento ipertestuale.

```
<a href="https://www.example.com">Visita Example</a>
```

Tag `<img>`

Inserisce un'immagine.

```

```

Attributo	Descrizione	Obbligatorio
<code>src</code>	Percorso o URL dell'immagine da visualizzare	Sì
<code>alt</code>	Testo alternativo per l'accessibilità e in caso di errore nel caricamento	Sì
<code>width</code>	Larghezza dell'immagine (in pixel o percentuale)	No
<code>height</code>	Altezza dell'immagine (in pixel o percentuale)	No
<code>title</code>	Testo visualizzato come tooltip al passaggio del mouse	No

Attributo	Descrizione	Obbligatorio
loading	Modalità di caricamento: lazy (differito) o eager (immediato)	No
srcset	Elenco di immagini alternative per dispositivi e risoluzioni diverse	No
sizes	Dimensioni da usare con srcset per immagini responsive	No
crossorigin	Gestione delle richieste CORS (anonymous , use-credentials)	No
referrerpolicy	Politica di invio del referrer	No
usemap	Collega una mappa immagine	No
ismap	Indica che l'immagine è una mappa lato server	No

Esempio:

```

```

Esercizio con il tag `<img>`

inserire un tag con un'immagine esistente e poi provare a inserire un'immagine errata per vedere l'importanza dell'attributo `alt`.

immagine errata

```

```

immagine esistente

```

```

Tag `<ul>`, `<ol>`, `<li>`

Liste non ordinate e ordinate.

```
<ul>
  <li>Elemento 1</li>
  <li>Elemento 2</li>
</ul>
<ol>
  <li>Primo</li>
  <li>Secondo</li>
</ol>
```


Tag `<table>`

Crea una tabella.

```
<table>
  <tr>
    <th>Nome</th>
    <th>Cognome</th>
  </tr>
  <tr>
    <td>Mario</td>
    <td>Rossi</td>
  </tr>
</table>
```

Esercizi HTML (valida il codice online o su vscode)

1. Crea una pagina HTML di base

Scrivi il codice per una pagina che contiene un titolo, un paragrafo e un'immagine.

2. Aggiungi un link

Inserisci un collegamento che porta a <https://www.wikipedia.org> - verificare online gli attributi del tag `<a>` e provare a utilizzare valori diversi per `target`.

3. Aggiungi una lista

Crea una lista non ordinata con almeno tre elementi e una lista ordinata con almeno due elementi.

4. Aggiungi una tabella semplice

Realizza una tabella con tre colonne (Nome, Età, Avatar) e tre righe di dati. Valorizzare la tabella con immagini per gli avatar e dei contenuti di esempio.

Tag `<form>`

Crea un modulo.

```
<form>  
  <input type="text" name="nome" />  
  <input type="submit" value="Invia" />  
</form>
```

Dettagli sul tag `<form>`

- Il tag `<form>` serve per raccogliere dati dagli utenti e inviarli a un server.
- Gli attributi principali sono:
 - `action` : URL a cui vengono inviati i dati.
 - `method` : metodo di invio (`get` o `post`).
 - `enctype` : tipo di codifica dei dati (es. `multipart/form-data` per upload di file).

Esempio con attributi:

```
<form action="/invia" method="post">
  <input type="text" name="nome" />
  <input type="submit" value="Invia" />
</form>
```

- All'interno del `<form>` si possono inserire vari campi: `<input>`, `<textarea>`, `<select>`, `<button>`, `<label>`, ecc.
- È buona pratica associare le etichette ai campi tramite l'attributo `for` di `<label>` e l'attributo `id` del campo.
- I dati inviati dal form possono essere elaborati da script lato server (es. PHP, Node.js, Python).

Consiglio: Verifica sempre che il form abbia un metodo e un'azione corretti per la sicurezza e la funzionalità.

Tag `<input>`

Campo di input.

```
<input type="text" placeholder="Inserisci testo" />
```

Attributi principali del tag `<input>`

Gli attributi più comuni e utili per il campo `<input>` sono:

- `type` — Tipo di campo (es: text, password, email, number, checkbox, radio, file, date, ecc.)
- `name` — Nome del campo (usato per l'invio dei dati)
- `id` — Identificatore univoco (utile per collegare una `<label>`)
- `value` — Valore predefinito del campo
- `placeholder` — Testo suggerito all'interno del campo
- `required` — Rende il campo obbligatorio
- `readonly` — Campo di sola lettura
- `disabled` — Campo disabilitato (non modificabile né inviato)

- `maxlength` — Numero massimo di caratteri inseribili
- `minlength` — Numero minimo di caratteri richiesti
- `min` / `max` — Valore minimo/massimo (per numeri, date, ecc.)
- `step` — Incremento consentito (per numeri, date, ecc.)
- `pattern` — Espressione regolare per la validazione del formato
- `autocomplete` — Suggerisce valori già usati (on/off o tipo di dato)
- `autofocus` — Il campo riceve automaticamente il focus all'apertura della pagina
- `checked` — Seleziona di default (solo per checkbox/radio)
- `multiple` — Permette selezioni multiple (solo per file/email)
- `size` — Larghezza visibile del campo (in caratteri)
- `form` — Associa l'input a un form specifico tramite id

- `list` — Collega un `<datalist>` per suggerimenti automatici
- `accept` — Tipi di file accettati (solo per `input type="file"`)
- `tabindex` — Ordine di tabulazione

Esempio:

```
<input
  type="email"
  name="user_email"
  id="email"
  placeholder="Inserisci la tua email"
  required
  maxlength="50"
  autocomplete="email"
  autofocus
/>
```

Tag `<button>`

Crea un pulsante.

```
<button>Clicca qui</button>
```

Dettagli sul tag `<button>`

- Il tag `<button>` crea un pulsante cliccabile che può essere usato per inviare form, eseguire script o attivare azioni sulla pagina.
- Può contenere testo, HTML o elementi come immagini e icone.

- Attributi principali:

- `type` : specifica il tipo di pulsante (`button` , `submit` , `reset`). Il valore predefinito è `submit` se il pulsante è dentro un form.
- `name` : nome del pulsante, utile per l'invio dei dati del form.
- `value` : valore inviato quando il form viene inviato.
- `disabled` : disabilita il pulsante.
- `autofocus` : il pulsante riceve il focus all'apertura della pagina.
- `form` : associa il pulsante a un form specifico tramite id.

Esempi:

```
<button type="button">Normale</button>  
<button type="submit">Invia</button>  
<button type="reset">Resetta</button>  
<button disabled>Disabilitato</button>  
<button>  
   Con icona  
</button>
```

- Il contenuto del pulsante può essere personalizzato con testo, immagini o HTML.
- Per accessibilità, aggiungi sempre un testo descrittivo all'interno del pulsante.

Esercizio: Form di autenticazione

Crea un form HTML per l'autenticazione che richieda username e password. Al click su "Accedi", il form deve inviare i dati tramite POST a un servizio di test online, ad esempio <https://httpbin.org/post>.

Esercizio:

```
<form action="https://httpbin.org/post" method="post">
  <label for="username">Username:</label>
  <input type="text" id="username" name="username" required />

  <label for="password">Password:</label>
  <input type="password" id="password" name="password" required />

  <button type="submit">Accedi</button>
</form>
```

- Usa l'attributo `required` per rendere obbligatori i campi.
- Puoi vedere la risposta del servizio di test dopo l'invio.
- **Consiglio:** Non inserire dati reali, è solo un esempio didattico.

Tag `<label>`

Etichetta per un campo modulo.

```
<label for="email">Email:</label> <input id="email" type="email" />
```

Tag `<select>`, `<option>`

Menu a tendina.

```
<select>  
  <option>Opzione 1</option>  
  <option>Opzione 2</option>  
</select>
```

Tag `<textarea>`

Area di testo multilinea.

```
<textarea rows="4" cols="50"></textarea>
```

Tag `<div>`

Contenitore generico.

```
<div>Contenuto</div>
```


Tag `<span>`

Contenitore inline.

```
<span>Testo evidenziato</span>
```

Tag `<strong>`, `<em>`

Testo in grassetto e corsivo.

```
<strong>Importante</strong> <em>Enfasi</em>
```

Tag `<header>`, `<footer>`

Intestazione e piè di pagina.

```
<header>Intestazione</header>  
<footer>Piè di pagina</footer>
```

Tag `<nav>`

Navigazione.

```
<nav>  
  <a href="#home">Home</a>  
  <a href="#contatti">Contatti</a>  
</nav>
```

Tag `<main>`

Contenuto principale.

```
<main>  
  <h1>Benvenuto</h1>  
</main>
```

Tag `<section>`

Sezione di contenuto.

```
<section>  
  <h2>Sezione 1</h2>  
</section>
```

Tag `<article>`

Articolo indipendente.

```
<article>  
  <h2>Notizia</h2>  
  <p>Testo della notizia.</p>  
</article>
```

Tag `<aside>`

Contenuto a lato.

```
<aside>  
  <p>Informazioni aggiuntive.</p>  
</aside>
```

Tag `<figure>`, `<figcaption>`

Immagine con didascalia.

```
<figure>  
    
  <figcaption>Didascalia</figcaption>  
</figure>
```


Tag `<iframe>`

Incorpora una pagina web.

```
<iframe src="https://www.example.com"></iframe>
```

Tag `<meta>`

Metadati della pagina.

```
<meta charset="UTF-8" />
```

Tag `<link>`

Collega risorse esterne.

```
<link rel="stylesheet" href="stile.css" />
```

Tag `<script>`

Inserisce codice JavaScript.

```
<script>  
  alert("Ciao!");  
</script>
```

Tag `<style>`

Stili CSS interni.

```
<style>
  body {
    background: #eee;
  }
</style>
```

Tag `<sup>`, `<sub>`

Apice e pedice.

```
H<sub>2</sub>O x<sup>2</sup>
```

Tag `<code>`, `<pre>`

Codice e testo preformattato.

```
<pre>  
  <code>console.log('Hello');</code>  
</pre>
```

Tag `<audio>`, `<video>`

Media audio e video.

```
<audio controls src="audio.mp3"></audio>  
<video controls src="video.mp4"></video>
```

Esempio di tag `<audio>`

```
<audio  
  controls  
  src="https://samplelib.com/lib/preview/mp3/sample-3s.mp3"  
></audio>
```


Esempio di tag `<video>`

```
<video controls width="320" height="240">  
  <source  
    src="https://samplelib.com/lib/preview/mp4/sample-5s.mp4"  
    type="video/mp4"  
  />  
  Il tuo browser non supporta il tag video.  
</video>
```

Tag `<details>`, `<summary>`

Sezione espandibile.

```
<details>  
  <summary>Mostra dettagli</summary>  
  Informazioni nascoste.  
</details>
```

Come impostare una pagina HTML

```
<!DOCTYPE html>
<html lang="it">
  <head>
    <meta charset="UTF-8" />
    <title>Titolo della Pagina</title>
  </head>
  <body>
    <!-- Contenuto visibile qui -->
  </body>
</html>
```

- **<!DOCTYPE html>** : indica al browser che si tratta di un documento HTML5.
- **<html lang="it">** : elemento radice, con attributo per la lingua.
- **<head>** : contiene metadati, titolo, link a risorse esterne.
- **<body>** : contiene tutto il contenuto visibile della pagina.

Esercizi HTML

1. Crea una pagina HTML con titolo e due paragrafi.
2. Inserisci un'immagine con attributo `alt` e una linea orizzontale (`<hr>`).
3. Aggiungi una lista non ordinata con almeno quattro elementi.
4. Crea una tabella con tre colonne (Nome, Cognome, Età) e due righe di dati.
5. Inserisci un link che si apre in una nuova scheda e un altro che punta a una sezione interna della pagina.
6. Crea un modulo con un campo di testo, una textarea e un pulsante di invio.
7. Aggiungi una sezione `<header>` con un titolo e una barra di navigazione `<nav>` con tre link.

8. Inserisci una struttura con `<main>`, `<section>`, `<article>` e `<aside>`, ognuno con contenuto testuale.
9. Crea una pagina con un `<figure>` che contiene un'immagine e una didascalia, e aggiungi un `<footer>` con un testo.
10. Incorpora un video e un audio nella pagina, aggiungi uno stile CSS interno che cambia il colore di sfondo del `<body>`.

Consiglio: Dopo ogni esercizio, valida il codice e verifica la compatibilità dei tag usati!

Altri elementi e concetti importanti in HTML

1. Commenti HTML

I commenti servono per annotare il codice e non vengono visualizzati dal browser.

```
<!-- Questo è un commento -->
```

2. Entità HTML

Le entità permettono di inserire caratteri speciali che altrimenti verrebbero interpretati come codice.

- `<` per `<`
- `>` per `>`
- `&` per `&`
- `"` per `"`
- `©` per `©`

3. Attributi globali

Questi attributi possono essere usati su quasi tutti i tag HTML:

- `id` : identificatore univoco
- `class` : una o più classi CSS
- `style` : stili CSS inline
- `title` : testo descrittivo (tooltip)
- `tabindex` : ordine di tabulazione
- `hidden` : nasconde l'elemento

4. Accessibilità

Scrivere HTML accessibile significa rendere le pagine utilizzabili da tutti, anche da persone con disabilità.

- Usa sempre `alt` per le immagini.
- Struttura i titoli in modo gerarchico (`<h1>` , `<h2>` , ...).
- Usa le etichette (`<label>`) per i campi modulo.
- Preferisci tag semantici (`<nav>` , `<main>` , `<footer>` , ecc.).

5. Tag semantici

I tag semantici descrivono il significato del contenuto, migliorando SEO e accessibilità:

- `<header>`, `<footer>`, `<nav>`, `<main>`, `<section>`, `<article>`, `<aside>`

6. Best practice

- Indenta il codice per renderlo leggibile.
- Chiudi sempre i tag (anche quelli opzionali, per chiarezza).
- Usa nomi significativi per `id` e `class`.
- Valida spesso il codice.

7. Struttura annidata

Gli elementi HTML possono essere annidati, ma bisogna rispettare la gerarchia e chiudere correttamente i tag.

```
<ul>  
  <li>Elemento <strong>importante</strong></li>  
</ul>
```

8. Responsive design

HTML lavora insieme a CSS per rendere le pagine adattabili a diversi dispositivi. Usa il tag

`<meta name="viewport" ...>` per migliorare la visualizzazione su mobile.

```
<meta name="viewport" content="width=device-width, initial-scale=1.0" />
```

Responsive design: dettagli e implementazione

Il **responsive design** permette alle pagine web di adattarsi automaticamente a schermi di dimensioni diverse (desktop, tablet, smartphone), migliorando l'esperienza utente.

Principi chiave

- **Layout fluido:** usa unità relative come `%`, `em`, `rem`, `vw`, `vh` invece di pixel fissi.
- **Media queries:** regole CSS che applicano stili diversi in base alla larghezza dello schermo.
- **Immagini responsive:** usa attributi come `srcset` e CSS `max-width: 100%` per adattare le immagini.
- **Tipografia scalabile:** dimensioni dei font che si adattano al dispositivo.

Esempio pratico

Aggiungi nel `<head>` :

```
<meta name="viewport" content="width=device-width, initial-scale=1.0" />
```

Esempio di media query CSS:

```
<style>
  body {
    font-size: 1.2em;
    background-color: #f00;
  }
  @media (max-width: 600px) {
    body {
      font-size: 1em;
      background-color: #0f0;
    }
  }
</style>
<p>Hello World!</p>
```

Best practice

- Progetta “mobile first”: scrivi prima gli stili per dispositivi piccoli, poi aggiungi media query per schermi più grandi.
- Testa la pagina su diversi dispositivi e simulatori.
- Usa strumenti come Chrome DevTools per simulare varie risoluzioni.

Risorse utili:

- [MDN Responsive design](#)
- [Google Web Fundamentals: Responsive Web Design Basics](#)

9. Collegamento tra file

Puoi collegare file CSS, JavaScript, immagini e altre risorse tramite i tag `<link>`, `<script>`, `<img>`, ecc.

10. Strumenti di ispezione

I browser moderni offrono strumenti per ispezionare e modificare il codice HTML in tempo reale (tasto destro → "Ispeziona").

Consiglio: Approfondisci questi concetti per scrivere pagine HTML più efficaci, accessibili e professionali!

Esercizi finali - Metti in pratica!

1. Crea una pagina HTML completa

Realizza una pagina che includa:

- Un `<header>` con titolo e navigazione
- Un `<main>` con almeno due `<section>`, una con un `<article>` e una con una tabella
- Un `<aside>` con informazioni aggiuntive
- Un `<footer>` con i tuoi contatti

2. Modulo di contatto

Inserisci un `<form>` con:

- Campi per nome, email, messaggio
- Etichette associate ai campi
- Un pulsante di invio
- Assicurati di usare gli attributi corretti per l'accessibilità

3. Media e stili

- Aggiungi un'immagine con didascalia usando `<figure>` e `<figcaption>`
- Incorpora un video o un audio
- Inserisci uno stile CSS interno che cambi il colore di sfondo e il colore dei titoli

4. Link e ancore

- Crea almeno due link esterni che si aprono in una nuova scheda
- Aggiungi un link interno che porta a una sezione specifica della pagina

5. Validazione e compatibilità

- Valida il tuo file HTML con il [W3C Validator](#)
- Controlla la compatibilità dei tag usati su [Can I use](#)

Sfida extra:

Prova a rendere la pagina responsive aggiungendo il tag `<meta name="viewport" ...>` e verifica come si adatta su dispositivi diversi.